

**S1. Tarea. Modelo conceptual de la red social estudiantil Pascualina.****Informe Final – Tarea 1**

Integrantes del equipo:

Karen Julieth Castro Arias

Jorge Eliecer Mazo Uparela

Samuel Hincapié Pérez

Paula Andrea Morales Zapata

Ali Jose Arias Basto

Curso: Bases de Datos I – G100

Oralia Cortes Grajales

Enlace al repositorio de GitHub: <https://github.com/Jorge-Mazo/bd1-redsocial-grupo6>

Enlace al video:

<https://drive.google.com/drive/folders/19cCMvVN69FymgJMTMs9yY4YnyEL26q4s?usp=sharing>

13 de noviembre de 2026

## **Tabla de contenido**

- 1. Introducción**
- 2. Análisis de requerimientos**
- 3. Metodología**
- 4. Entidades identificadas y justificación**
- 5. Atributos de cada entidad**
- 6. Relaciones, cardinalidades y participación**
- 7. Rompimiento del modelo lógico**
- 8. Diagrama Entidad-Relación final**
- 9. Jerarquías o generalizaciones**
- 10. Conclusiones individuales**
- 11. Referencias**
- 12. Aportes Individuales**

## **1. Introducción**

La Red Social Estudiantil Pascualina surge como respuesta a una necesidad identificada en la comunidad educativa: mejorar la comunicación y la interacción entre los estudiantes. Aunque la institución cuenta con plataformas institucionales, estas son percibidas como formales y no facilitan la conexión casual, el intercambio de conocimientos entre pares, la formación de grupos de estudio ni la organización de eventos informales.

El presente informe documenta el diseño del modelo conceptual de la base de datos que soportará esta red social, aplicando el Modelo Entidad-Relación (MER) y el rompimiento del modelo lógico desde la etapa conceptual. El objetivo es entregar un modelo que permita a los estudiantes crear perfiles, conectar con compañeros, publicar actualizaciones, compartir recursos académicos, crear y unirse a grupos, programar eventos, gestionar mentorías y comunicarse internamente.

El documento se organiza en doce secciones que cubren el análisis de requerimientos, la identificación y justificación de entidades, la definición de atributos, el establecimiento de relaciones con cardinalidades y participación, el rompimiento del modelo lógico con las entidades intermedias y llaves foráneas, el diagrama final, las jerarquías evaluadas, las conclusiones individuales del equipo, las referencias y los aportes individuales.

## **2. Análisis de requerimientos**

### **2.1 Contextualización del reto**

La institución Pascual Bravo ha identificado que sus plataformas actuales no favorecen la interacción informal entre estudiantes. Se requiere una red social que permita:

- Crear perfiles con área de estudio, intereses y habilidades.
- Conectar con compañeros con intereses similares o de cursos avanzados que ofrezcan mentoría.
- Publicar actualizaciones: preguntas, recursos, noticias y memes.
- Compartir recursos académicos formales.
- Crear y unirse a grupos de estudio, hackatones o clubes.
- Programar eventos: reuniones, talleres y actividades sociales.
- Gestionar sesiones de mentoría entre estudiantes avanzados y aprendices.
- Comunicarse internamente mediante mensajería.

### **2.2 Requerimientos funcionales**

#### ***Módulo 1: Usuarios y datos académicos***

RF-01. La plataforma debe permitir registrar estudiantes con identificador único, nombres, apellidos, correo institucional, fecha de nacimiento, semestre y fecha de registro.

RF-02. Cada estudiante debe estar asociado a un programa académico.

RF-03. Cada programa académico debe tener materias o asignaturas asociadas.

RF-04. Cada estudiante debe tener un perfil social con descripción personal, área de estudio, fotografía, biografía y tipo de mentoría.

### ***Módulo 2: Mentorías e intercambio académico***

RF-05. La plataforma debe permitir que un estudiante avanzado se habilite como mentor de una o varias materias.

RF-06. La plataforma debe permitir agendar sesiones de mentoría entre un mentor y un aprendiz, con fecha, hora, tema y estado.

### ***Módulo 3: Perfiles y preferencias***

RF-07. La plataforma debe gestionar un catálogo de intereses.

RF-08. La plataforma debe gestionar un catálogo de habilidades.

RF-09. Un estudiante puede registrar uno o varios intereses.

RF-10. Un estudiante puede registrar una o varias habilidades.

### ***Módulo 4: Publicaciones e interacciones sociales***

RF-11. La plataforma debe gestionar un catálogo de tipos de publicación.

RF-12. Un estudiante puede crear publicaciones en el muro.

RF-13. Un estudiante puede seguir a otros estudiantes y ser seguido por ellos.

***Módulo 5: Recursos académicos***

RF-14. La plataforma debe gestionar un catálogo de tipos de recurso.

RF-15. Un estudiante puede compartir recursos académicos asociados a una materia.

RF-16. Un estudiante puede valorar los recursos académicos compartidos por otros.

***Módulo 6: Comunidades, grupos y eventos***

RF-17. La plataforma debe gestionar un catálogo de tipos de grupo.

RF-18. Un estudiante puede crear grupos y unirse a grupos existentes.

RF-19. Un estudiante puede crear eventos y participar en eventos de otros.

RF-20. La plataforma debe gestionar la asistencia a eventos.

***Módulo 7: Comunicación interna***

RF-21. La plataforma debe permitir el envío de mensajes directos entre estudiantes.

**2.3 Requerimientos de integridad**

RI-01. Toda publicación debe tener un autor obligatorio.

RI-02. Todo grupo debe tener un creador obligatorio.

RI-03. Todo evento debe tener un creador obligatorio.

RI-04. Todo perfil debe estar asociado a un único estudiante.

RI-05. Todo estudiante debe pertenecer a un único programa académico.

RI-06. Toda relación de muchos a muchos debe resolverse mediante una entidad intermedia con llaves foráneas.

RI-07. Toda sesión de mentoría debe tener un mentor, un aprendiz y una materia.

RI-08. Todo recurso académico debe estar asociado a una materia y a un autor.

RI-09. Todo mensaje debe tener un emisor y un receptor.

## **2.4 Justificación del alcance**

Se modelaron veintidós entidades organizadas en siete módulos. El alcance responde directamente a los requerimientos del reto y a las necesidades de una red social estudiantil completa. Se incluyeron catálogos maestros para normalizar tipos y estados, entidades transaccionales para las acciones del sistema y entidades intermedias para el rompimiento del modelo lógico.

## **3. Metodología**

### **3.1 Enfoque de trabajo**

El equipo aplicó una metodología en cinco etapas:

1. Análisis individual de las necesidades del reto.
2. Lluvia de ideas grupal y construcción de un mapa conceptual inicial.
3. Elaboración de diagramas MER individuales por cada integrante.
4. Presentación, comparación y retroalimentación entre pares.
5. Consolidación del modelo final y elaboración del informe.

### 3.2 Herramientas utilizadas

- draw.io y PlantUML para los diagramas.
- GitHub para la gestión de versiones.
- Google Meet para las reuniones.
- Word y Markdown para la redacción del informe.

### 3.3 Aplicación del rompimiento del modelo lógico

Aunque se trata de un modelo conceptual, la docente solicitó aplicar el rompimiento del modelo lógico desde esta etapa. Esto implica que las relaciones de muchos a muchos se transformaron en entidades intermedias con llaves foráneas y llave primaria compuesta. Las relaciones de uno a muchos colocaron la llave foránea en el lado muchos. Las relaciones reflexivas se resolvieron con entidades intermedias de dos llaves foráneas que referencian a la misma entidad con roles distintos.

## 4. Entidades identificadas y justificación

### 4.1 Módulo 1: Usuarios y datos académicos

PROGRAMA\_ACADEMICO. Entidad maestra o catálogo. Almacena las carreras ofrecidas por la institución, por ejemplo, Tecnología en Desarrollo de Software, Ingeniería Eléctrica o Medicina. Es entidad y no atributo porque agrupa a muchos estudiantes y tiene existencia independiente. Sus atributos propios son el código y el nombre del programa.

MATERIA\_ASIGNATURA. Entidad maestra o catálogo. Almacena las materias específicas asociadas a cada programa, por ejemplo, Bases de Datos I, Anatomía o Cálculo Diferencial. Es entidad porque se repite entre estudiantes, grupos de estudio, recursos y mentorías. Tiene atributos propios y se relaciona con el programa académico.

ESTUDIANTE. Entidad maestra o core. Representa al actor central de la red social. Contiene los datos institucionales del alumno. Es la entidad de la que dependen la mayoría de las relaciones del modelo.

PERFIL\_SOCIAL. Entidad maestra o extensión. Contiene los datos complementarios de la red social: biografía, fotografía, área de estudio y tipo de mentoría. Se relaciona uno a uno con Estudiante. Se separa porque sus atributos son específicos de la vida social en la plataforma y no forman parte de la identidad institucional.

#### **4.2 Módulo 2: Mentorías e intercambio académico**

SESION\_MENTORIA. Entidad transaccional. Controla las citas, estados y temas de las asesorías académicas entre estudiantes avanzados y solicitantes. Tiene atributos propios como fecha, hora, tema y estado.

MENTOR\_MATERIA\_HABILITADA. Entidad de rompimiento de la relación de muchos a muchos entre Estudiante y Materia. Cruza qué estudiante avanzado está habilitado para dar tutoría de qué materia específica.

#### **4.3 Módulo 3: Perfiles y preferencias**

INTERES. Entidad maestra o catálogo. Almacena las temáticas generales de interés, por ejemplo, Inteligencia Artificial, Fútbol o Videojuegos. Es entidad porque se repite entre estudiantes y debe poder administrarse de forma independiente.

HABILIDAD. Entidad maestra o catálogo. Almacena las aptitudes o destrezas, por ejemplo, Programación en Python, Oratoria o Diseño Gráfico. Igual que los intereses, se repite entre estudiantes y tiene atributos propios.

ESTUDIANTE\_INTERES. Entidad de rompimiento. Vincula los múltiples intereses que puede seleccionar un estudiante. Su llave primaria es compuesta.

ESTUDIANTE\_HABILIDAD. Entidad de rompimiento. Vincula las múltiples habilidades que posee un estudiante. Su llave primaria es compuesta.

#### **4.4 Módulo 4: Publicaciones e interacciones sociales**

TIPO\_PUBLICACION. Entidad maestra o catálogo. Clasifica las publicaciones en pregunta, recurso útil, noticia o meme. Permite normalizar el atributo tipo en lugar de usar texto libre.

PUBLICACION. Entidad transaccional. Representa el muro de actualizaciones con el contenido y los archivos o código adjuntos que los estudiantes comparten.

SEGUIDOR\_ESTUDIANTE. Entidad de rompimiento de la relación reflexiva de muchos a muchos entre Estudiante y Estudiante. Gestiona la red de contactos de la plataforma: quién sigue a quién.

#### **4.5 Módulo 5: Recursos académicos**

TIPO\_RECURSO. Entidad maestra o catálogo. Clasifica los recursos en PDF, enlace, código fuente, video o documento.

RECURSO\_ACADEMICO. Entidad transaccional. Representa el material, documento, enlace o código fuente que un estudiante comparte formalmente para ayudar a sus pares. Se asocia a una materia y a un autor.

VALORACION\_RECURSO. Entidad de rompimiento de la relación de muchos a muchos entre Estudiante y Recurso\_Academico. Almacena la puntuación y el comentario que un estudiante deja sobre un recurso.

#### **4.6 Módulo 6: Comunidades, grupos y eventos**

TIPO\_GRUPO. Entidad maestra o catálogo. Clasifica los grupos en grupo de estudio, equipo para hackatón o club de interés.

GRUPO. Entidad maestra o estructural. Representa los espacios comunitarios creados por los estudiantes. Tiene un creador obligatorio y varios miembros.

EVENTO. Entidad transaccional. Representa talleres, reuniones o actividades sociales agendadas. Tiene un creador y varios participantes.

MIEMBRO\_GRUPO. Entidad de rompimiento de la relación de muchos a muchos entre Estudiante y Grupo. Controla la pertenencia y el rol dentro del grupo: miembro o moderador.

ASISTENCIA\_EVENTO. Entidad de rompimiento de la relación de muchos a muchos entre Estudiante y Evento. Gestiona los inscritos y su estado de asistencia.

#### **4.7 Módulo 7: Comunicación interna**

MENSAJE. Entidad de rompimiento de la relación reflexiva de muchos a muchos entre Estudiante y Estudiante. Almacena los mensajes directos con emisor, receptor, contenido, fecha y estado de lectura.

## **5. Atributos de cada entidad**

Convenciones: PK significa clave primaria, FK significa clave foránea, el asterisco indica obligatorio y el guion indica opcional.

### **5.1 Módulo 1: Usuarios y datos académicos**

#### **PROGRAMA\_ACADEMICO**

- codigo\_programa: PK, obligatorio. Identificador único del programa.
- nombre\_programa: texto, obligatorio. Nombre de la carrera.
- descripcion: texto, opcional. Descripción del programa.
- nivel: texto, obligatorio. Técnico, tecnológico o profesional.

Justificación: los atributos permiten identificar y describir el programa. El nivel permite clasificar el tipo de formación.

#### **MATERIA\_ASIGNATURA**

- codigo\_materia: PK, obligatorio. Identificador único de la materia.
- nombre\_materia: texto, obligatorio. Nombre de la materia.
- creditos: numérico, obligatorio. Número de créditos.
- semestre: numérico, obligatorio. Semestre en que se cursa.

- codigo\_programa: FK, obligatorio. Referencia a PROGRAMA\_ACADEMICO(codigo\_programa).

Justificación: cada materia pertenece a un programa. La llave foránea va en el lado muchos, es decir, en MATERIA\_ASIGNATURA.

## **ESTUDIANTE**

- codigo\_estudiante: PK, obligatorio. Identificador único del estudiante.
- nombre: texto, obligatorio. Nombres.
- apellido: texto, obligatorio. Apellidos.
- correo\_institucional: texto, obligatorio. Correo del Pascual.
- fecha\_nacimiento: fecha, obligatorio. Fecha de nacimiento.
- semestre: numérico, obligatorio. Semestre que cursa.
- fecha\_registro: fecha, obligatorio. Fecha de registro en la plataforma.
- codigo\_programa: FK, obligatorio. Referencia a PROGRAMA\_ACADEMICO(codigo\_programa).

Justificación: el estudiante pertenece a un programa. La llave foránea se ubica aquí porque es el lado muchos de la relación.

## **PERFIL\_SOCIAL**

- codigo\_perfil: PK, obligatorio. Identificador único del perfil.
- descripcion\_personal: texto, opcional. Descripción personal.

- area\_estudio: texto, obligatorio. Área de estudio principal.
- fotografia: texto, opcional. URL o ruta de la fotografía.
- biografia: texto, opcional. Biografía del estudiante.
- tipo\_mentoria: texto, opcional. Tipo de mentoría que ofrece.
- codigo\_estudiante: FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).

Justificación: relación uno a uno con Estudiante. La llave foránea se ubica en el perfil, que es el lado dependiente.

## 5.2 Módulo 2: Mentorías e intercambio académico

### SESION\_MENTORIA

- codigo\_sesion: PK, obligatorio. Identificador único de la sesión.
- fecha\_sesion: fecha, obligatorio. Fecha de la asesoría.
- hora\_sesion: hora, obligatorio. Hora de la asesoría.
- tema: texto, obligatorio. Tema tratado.
- estado: texto, obligatorio. Programada, realizada o cancelada.
- codigo\_mentor: FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).
- codigo\_aprendiz: FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).
- codigo\_materia: FK, obligatorio. Referencia a MATERIA\_ASIGNATURA(codigo\_materia).

Justificación: la sesión relaciona al mentor, al aprendiz y la materia. Las tres llaves foráneas permiten identificar la transacción completa.

### **MENTOR\_MATERIA\_HABILITADA**

- `codigo_estudiante`: PK, FK, obligatorio. Referencia a `ESTUDIANTE(codigo_estudiante)`.
- `codigo_materia`: PK, FK, obligatorio. Referencia a `MATERIA_ASIGNATURA(codigo_materia)`.
- `fecha_habilitacion`: fecha, obligatorio. Fecha de habilitación.
- `certificado`: texto, opcional. Evidencia o certificado.

Justificación: resuelve la relación de muchos a muchos entre Estudiante y Materia. La llave primaria es compuesta.

## **5.3 Módulo 3: Perfiles y preferencias**

### **INTERES**

- `codigo_interes`: PK, obligatorio. Identificador único del interés.
- `nombre_interes`: texto, obligatorio. Nombre del interés.
- `descripcion`: texto, opcional. Descripción del interés.

### **HABILIDAD**

- `codigo_habilidad`: PK, obligatorio. Identificador único de la habilidad.
- `nombre_habilidad`: texto, obligatorio. Nombre de la habilidad.
- `descripcion`: texto, opcional. Descripción de la habilidad.

### **ESTUDIANTE\_INTERES**

- `codigo_estudiante`: PK, FK, obligatorio. Referencia a `ESTUDIANTE(codigo_estudiante)`.
- `codigo_interes`: PK, FK, obligatorio. Referencia a `INTERES(codigo_interes)`.
- `fecha_asociacion`: fecha, opcional. Fecha de asociación.

#### **ESTUDIANTE\_HABILIDAD**

- `codigo_estudiante`: PK, FK, obligatorio. Referencia a `ESTUDIANTE(codigo_estudiante)`.
- `codigo_habilidad`: PK, FK, obligatorio. Referencia a `HABILIDAD(codigo_habilidad)`.
- `nivel`: texto, opcional. Nivel de dominio.

### **5.4 Módulo 4: Publicaciones e interacciones sociales**

#### **TIPO\_PUBLICACION**

- `codigo_tipo_publicacion`: PK, obligatorio. Identificador del tipo.
- `nombre_tipo`: texto, obligatorio. Pregunta, recurso, noticia o meme.
- `descripcion`: texto, opcional. Descripción del tipo.

#### **PUBLICACION**

- `codigo_publicacion`: PK, obligatorio. Identificador único.
- `contenido`: texto, obligatorio. Contenido de la publicación.
- `fecha_publicacion`: fecha, obligatorio. Fecha de publicación.

- `archivo_adjunto`: texto, opcional. URL o ruta del adjunto.
- `codigo_estudiante`: FK, obligatorio. Referencia a `ESTUDIANTE(codigo_estudiante)`.
- `codigo_tipo_publicacion`: FK, obligatorio. Referencia a `TIPO_PUBLICACION(codigo_tipo_publicacion)`.

### **SEGUIDOR\_ESTUDIANTE**

- `codigo_seguidor`: PK, FK, obligatorio. Referencia a `ESTUDIANTE(codigo_estudiante)`.
- `codigo_seguido`: PK, FK, obligatorio. Referencia a `ESTUDIANTE(codigo_estudiante)`.
- `fecha_seguimiento`: fecha, obligatorio. Fecha en que comenzó el seguimiento.

Justificación: resuelve la relación reflexiva. La llave primaria es compuesta y las dos llaves foráneas referencian a la misma entidad con roles distintos.

## **5.5 Módulo 5: Recursos académicos**

### **TIPO\_RECURSO**

`codigo_tipo_recurso`: PK, obligatorio. Identificador del tipo.

`nombre_tipo`: texto, obligatorio. PDF, enlace, código, video o documento.

descripcion: texto, opcional. Descripción del tipo.

### **RECURSO\_ACADEMICO**

- codigo\_recurso: PK, obligatorio. Identificador único.
- titulo: texto, obligatorio. Título del recurso.
- descripcion: texto, opcional. Descripción del recurso.
- url\_archivo: texto, obligatorio. URL o ruta del recurso.
- fecha\_subida: fecha, obligatorio. Fecha de carga.
- codigo\_estudiante: FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).
- codigo\_materia: FK, obligatorio. Referencia a

### **MATERIA\_ASIGNATURA(codigo\_materia).**

- codigo\_tipo\_recurso: FK, obligatorio. Referencia a TIPO\_RECURSO(codigo\_tipo\_recurso).

### **VALORACION\_RECURSO**

- codigo\_estudiante: PK, FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).
- codigo\_recurso: PK, FK, obligatorio. Referencia a RECURSO\_ACADEMICO(codigo\_recurso).
- puntuacion: numérico, obligatorio. Valor de uno a cinco.

- comentario: texto, opcional. Comentario sobre el recurso.
- fecha\_valoracion: fecha, obligatorio. Fecha de la valoración.

## 5.6 Módulo 6: Comunidades, grupos y eventos

### TIPO\_GRUPO

- codigo\_tipo\_grupo: PK, obligatorio. Identificador del tipo.
- nombre\_tipo: texto, obligatorio. Estudio, hackatón o club.
- descripcion: texto, opcional. Descripción del tipo.

### GRUPO

- codigo\_grupo: PK, obligatorio. Identificador único.
- nombre\_grupo: texto, obligatorio. Nombre del grupo.
- descripcion: texto, opcional. Descripción del grupo.
- fecha\_creacion: fecha, obligatorio. Fecha de creación.
- codigo\_estudiante\_creador: FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).
- codigo\_tipo\_grupo: FK, obligatorio. Referencia a TIPO\_GRUPO(codigo\_tipo\_grupo).

### EVENTO

- codigo\_evento: PK, obligatorio. Identificador único.
- nombre\_evento: texto, obligatorio. Nombre del evento.

- fecha\_evento: fecha, obligatorio. Fecha del evento.
  - hora\_evento: hora, obligatorio. Hora del evento.
  - lugar: texto, obligatorio. Lugar del evento.
- 
- estado: texto, obligatorio. Programado, realizado o cancelado.
  - descripcion: texto, opcional. Descripción del evento.
  - codigo\_estudiante\_creador: FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).

#### **MIEMBRO\_GRUPO**

- codigo\_estudiante: PK, FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).
- codigo\_grupo: PK, FK, obligatorio. Referencia a GRUPO(codigo\_grupo).
- fecha\_union: fecha, obligatorio. Fecha de ingreso.
- rol: texto, obligatorio. Miembro o moderador.

#### **ASISTENCIA\_EVENTO**

- codigo\_estudiante: PK, FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).
- codigo\_evento: PK, FK, obligatorio. Referencia a EVENTO(codigo\_evento).
- fecha\_inscripcion: fecha, obligatorio. Fecha de inscripción.
- estado\_asistencia: texto, obligatorio. Inscrito, asistió o no asistió.

## 5.7 Módulo 7: Comunicación interna

### MENSAJE

- `codigo_mensaje`: PK, obligatorio. Identificador único.
- `contenido`: texto, obligatorio. Contenido del mensaje.
- `fecha_envio`: fecha, obligatorio. Fecha y hora del envío.
- `leido`: booleano, obligatorio. Indica si fue leído.
- `codigo_emisor`: FK, obligatorio. Referencia a ESTUDIANTE(`codigo_estudiante`).
- `codigo_receptor`: FK, obligatorio. Referencia a ESTUDIANTE(`codigo_estudiante`).

Justificación: resuelve la relación reflexiva de muchos a muchos entre Estudiante y Estudiante. Las dos llaves foráneas referencian a la misma entidad con roles distintos.

## 6. Relaciones, cardinalidades y participación

### 6.1 Relaciones del Módulo 1

PROGRAMA\_ACADEMICO tiene MATERIA\_ASIGNATURA. Cardinalidad uno a muchos. Participación: Programa puede, Materia debe. Rompimiento: la llave foránea `codigo_programa` se ubica en MATERIA\_ASIGNATURA.

PROGRAMA\_ACADEMICO tiene ESTUDIANTE. Cardinalidad uno a muchos. Participación: Programa puede, Estudiante debe. Rompimiento: la llave foránea `codigo_programa` se ubica en ESTUDIANTE.

ESTUDIANTE tiene PERFIL\_SOCIAL. Cardinalidad uno a uno. Participación: Estudiante debe, Perfil debe. Rompimiento: la llave foránea codigo\_estudiante se ubica en PERFIL\_SOCIAL.

## 6.2 Relaciones del Módulo 2

ESTUDIANTE (mentor) participa en SESION\_MENTORIA. Cardinalidad uno a muchos. Participación: Estudiante puede, Sesión debe. Rompimiento: la llave foránea codigo\_mentor se ubica en SESION\_MENTORIA.

ESTUDIANTE (aprendiz) participa en SESION\_MENTORIA. Cardinalidad uno a muchos. Participación: Estudiante puede, Sesión debe. Rompimiento: la llave foránea codigo\_aprendiz se ubica en SESION\_MENTORIA.

MATERIA\_ASIGNATURA se dicta en SESION\_MENTORIA. Cardinalidad uno a muchos. Participación: Materia puede, Sesión debe. Rompimiento: la llave foránea codigo\_materia se ubica en SESION\_MENTORIA.

ESTUDIANTE se habilita en MENTOR\_MATERIA\_HABILITADA. Cardinalidad uno a muchos. Participación: Estudiante puede, Habilitación debe. Rompimiento: la llave foránea codigo\_estudiante se ubica en MENTOR\_MATERIA\_HABILITADA.

MATERIA\_ASIGNATURA se habilita en MENTOR\_MATERIA\_HABILITADA. Cardinalidad uno a muchos. Participación: Materia puede, Habilitación debe. Rompimiento: la llave foránea codigo\_materia se ubica en MENTOR\_MATERIA\_HABILITADA.

## 6.3 Relaciones del Módulo 3

ESTUDIANTE tiene INTERES. Cardinalidad muchos a muchos. Participación:  
Estudiante puede, Interés puede. Rompimiento: entidad intermedia ESTUDIANTE\_INTERES.

ESTUDIANTE posee HABILIDAD. Cardinalidad muchos a muchos. Participación:  
Estudiante puede, Habilidad puede. Rompimiento: entidad intermedia  
ESTUDIANTE\_HABILIDAD.

#### **6.4 Relaciones del Módulo 4**

TIPO\_PUBLICACION clasifica PUBLICACION. Cardinalidad uno a muchos.  
Participación: Tipo puede, Publicación debe. Rompimiento: la llave foránea  
codigo\_tipo\_publicacion se ubica en PUBLICACION.

ESTUDIANTE publica PUBLICACION. Cardinalidad uno a muchos. Participación:  
Estudiante puede, Publicación debe. Rompimiento: la llave foránea codigo\_estudiante se ubica  
en PUBLICACION.

ESTUDIANTE sigue a ESTUDIANTE. Cardinalidad muchos a muchos. Participación:  
Estudiante puede, Estudiante puede. Rompimiento: entidad intermedia  
SEGUIDOR\_ESTUDIANTE con dos llaves foráneas.

#### **6.5 Relaciones del Módulo 5**

TIPO\_RECURSO clasifica RECURSO\_ACADEMICO. Cardinalidad uno a muchos.  
Participación: Tipo puede, Recurso debe. Rompimiento: la llave foránea codigo\_tipo\_recurso se  
ubica en RECURSO\_ACADEMICO.

ESTUDIANTE comparte RECURSO\_ACADEMICO. Cardinalidad uno a muchos.

Participación: Estudiante puede, Recurso debe. Rompimiento: la llave foránea

codigo\_estudiante se ubica en RECURSO\_ACADEMICO.

MATERIA\_ASIGNATURA tiene RECURSO\_ACADEMICO. Cardinalidad uno a muchos.

Participación: Materia puede, Recurso debe. Rompimiento: la llave foránea codigo\_materia se

ubica en RECURSO\_ACADEMICO.

ESTUDIANTE valora RECURSO\_ACADEMICO. Cardinalidad muchos a muchos.

Participación: Estudiante puede, Recurso puede. Rompimiento: entidad intermedia

VALORACION\_RECURSO.

## **6.6 Relaciones del Módulo 6**

TIPO\_GRUPO clasifica GRUPO. Cardinalidad uno a muchos. Participación: Tipo puede,

Grupo debe. Rompimiento: la llave foránea codigo\_tipo\_grupo se ubica en GRUPO.

ESTUDIANTE crea GRUPO. Cardinalidad uno a muchos. Participación: Estudiante

puede, Grupo debe. Rompimiento: la llave foránea codigo\_estudiante\_creador se ubica en

GRUPO.

ESTUDIANTE se une a GRUPO. Cardinalidad muchos a muchos. Participación:

Estudiante puede, Grupo puede. Rompimiento: entidad intermedia MIEMBRO\_GRUPO.

ESTUDIANTE crea EVENTO. Cardinalidad uno a muchos. Participación: Estudiante

puede, Evento debe. Rompimiento: la llave foránea codigo\_estudiante\_creador se ubica en

EVENTO.

ESTUDIANTE participa en EVENTO. Cardinalidad muchos a muchos. Participación: Estudiante puede, Evento puede. Rompimiento: entidad intermedia ASISTENCIA\_EVENTO.

## **6.7 Relaciones del Módulo 7**

ESTUDIANTE envía MENSAJE a ESTUDIANTE. Cardinalidad muchos a muchos. Participación: Estudiante puede, Estudiante puede. Rompimiento: entidad intermedia MENSAJE con dos llaves foráneas.

# **7. Rompimiento del modelo lógico**

## **7.1 Explicación del proceso**

El rompimiento del modelo lógico convierte las relaciones de muchos a muchos en entidades intermedias con llaves foráneas y llave primaria compuesta. Las relaciones de uno a muchos colocan la llave foránea en el lado muchos. Las relaciones reflexivas generan entidades intermedias con dos llaves foráneas que referencian a la misma entidad con roles distintos.

## **7.2 Entidades intermedias resultantes**

ESTUDIANTE\_INTERES. Llaves foráneas: codigo\_estudiante, codigo\_interes. Llave primaria compuesta. Atributo adicional: fecha\_asociacion. Justificación: resuelve la relación de muchos a muchos entre Estudiante e Interes.

ESTUDIANTE\_HABILIDAD. Llaves foráneas: codigo\_estudiante, codigo\_habilidad. Llave primaria compuesta. Atributo adicional: nivel. Justificación: resuelve la relación de muchos a muchos entre Estudiante y Habilidad.

SEGUIDOR\_ESTUDIANTE. Llaves foráneas: codigo\_seguidor, codigo\_seguido. Llave primaria compuesta. Atributo adicional: fecha\_seguimiento. Justificación: resuelve la relación reflexiva de muchos a muchos entre Estudiante y Estudiante.

MIEMBRO\_GRUPO. Llaves foráneas: codigo\_estudiante, codigo\_grupo. Llave primaria compuesta. Atributos adicionales: fecha\_union, rol. Justificación: resuelve la relación de muchos a muchos entre Estudiante y Grupo.

ASISTENCIA\_EVENTO. Llaves foráneas: codigo\_estudiante, codigo\_evento. Llave primaria compuesta. Atributos adicionales: fecha\_inscripcion, estado\_asistencia. Justificación: resuelve la relación de muchos a muchos entre Estudiante y Evento.

MENTOR\_MATERIA\_HABILITADA. Llaves foráneas: codigo\_estudiante, codigo\_materia. Llave primaria compuesta. Atributos adicionales: fecha\_habilitacion, certificado. Justificación: resuelve la relación de muchos a muchos entre Estudiante y Materia.

VALORACION\_RECURSO. Llaves foráneas: codigo\_estudiante, codigo\_recurso. Llave primaria compuesta. Atributos adicionales: puntuacion, comentario, fecha\_valoracion. Justificación: resuelve la relación de muchos a muchos entre Estudiante y Recurso\_Academico.

MENSAJE. Llaves foráneas: codigo\_emisor, codigo\_receptor. Llave primaria compuesta. Atributos adicionales: contenido, fecha\_envio, leído. Justificación: resuelve la relación reflexiva de muchos a muchos entre Estudiante y Estudiante.

### **7.3 Llaves foráneas en relaciones de uno a muchos**



Ubicación en el repositorio: tarea1/modelo-final/diagrama-mer-final.png

Simbología utilizada:

- Línea vertical doble: uno y solo uno.
- Pata de cuervo: muchos.
- Círculo: cero o opcional.
- Línea con pata: uno o muchos.
- Círculo con pata: cero o muchos.

## 9. Jerarquías o generalizaciones

Se evaluó la posibilidad de aplicar jerarquías en tres entidades.

PUBLICACION. Podría tener subtipos Pregunta, Recurso, Noticia y Meme. Decisión: no se modela como jerarquía porque el tipo se representa con el atributo `codigo_tipo_publicacion`, que referencia al catálogo TIPO\_PUBLICACION.

EVENTO. Podría tener subtipos Estudio, Taller y Social. Decisión: no se modela como jerarquía porque el tipo se puede representar con un atributo de tipo, y no aporta valor al modelo conceptual.

GRUPO. Podría tener subtipos Estudio, Hackatón y Club. Decisión: no se modela como jerarquía porque el atributo `codigo_tipo_grupo` referencia al catálogo TIPO\_GRUPO y cubre la necesidad.

En conclusión, no se aplicaron jerarquías porque los tipos se resuelven con catálogos y llaves foráneas, lo que simplifica el modelo y facilita su implementación.

### **Conclusión individual de Jorge Eliecer Mazo Uparela**

Mi principal dificultad fue comprender el rompimiento del modelo lógico desde la etapa conceptual. Al principio no entendía por qué las relaciones de muchos a muchos debían transformarse en entidades intermedias, ni cómo ubicar correctamente las llaves foráneas. A medida que avanzaba, aprendí que este proceso es esencial para garantizar la integridad y evitar redundancias. Comprendí que las relaciones de muchos a muchos requieren una entidad intermedia con llave primaria compuesta, y que en las relaciones de uno a muchos la llave foránea siempre se coloca en el lado muchos. Este aprendizaje me permitió aplicar correctamente el rompimiento en las veintidós entidades del modelo y entender la importancia de un buen diseño antes de implementar la base de datos. Además, el trabajo en equipo y la retroalimentación de mis compañeros fueron clave para superar las dudas y consolidar el modelo final.

### **Conclusión individual Samuel Hincapie Perez**

Mi aporte principal en esta fase fue revisar y organizar el modelo, tratando de entender bien cómo funcionaba cada entidad y cómo se relacionaba con las demás. Lo que más se me dificultó fue analizar todas las entidades, ya que eran varias y cada una tiene diferentes atributos y relaciones. Las relaciones también me costaron un poco, especialmente al momento de identificar cuáles necesitaban tablas intermedias y cuáles podían manejarse con claves

foráneas. Sin embargo, al revisar el diagrama y comparar las relaciones, pude entender mejor cómo se debía estructurar el modelo y cómo evitar errores en su diseño.

### **Conclusión Individual Paula Andrea Morales Zapata**

Uno de los principales aprendizajes fue entender que cada relación debe tener un propósito y que las cardinalidades permiten representar de forma más precisa la manera en que los usuarios interactúan con la plataforma. También fue importante reconocer que una buena estructura de datos ayuda a evitar información repetida y facilita que posteriormente la base de datos pueda ser implementada de manera organizada. Además considero que este conocimiento es fundamental para el desarrollo de sistemas y aplicaciones, ya que una base de datos bien diseñada facilita el manejo de la información y contribuye al correcto funcionamiento de cualquier plataforma.

### **Conclusión individual: Ali Jose Arias Basto**

Mi trabajo en esta fase se centró en revisar y organizar el modelo, entendiendo el funcionamiento de cada entidad y su relación con las demás. La mayor dificultad fue analizar el conjunto completo de entidades, ya que eran varias y cada una maneja atributos y relaciones distintas. El punto que más me costó fue identificar cuáles relaciones requerían una entidad intermedia y cuáles podían resolverse con una llave foránea; con la práctica comprendí que las relaciones de muchos a muchos exigen una tabla intermedia con llave primaria compuesta, mientras que las de uno a muchos se resuelven ubicando la llave foránea en el lado muchos. Comparar el diagrama con las relaciones definidas me permitió afianzar este criterio y

reconocer la importancia de estructurar bien el modelo desde la etapa conceptual para evitar errores en su diseño e implementación.

### **11. Referencias**

Cortes Grajales, O. (2026). Síntesis foro modelo conceptual Entidad Relación. Curso Bases de Datos I, G100 (SD1006). Tecnología en Desarrollo de Software.

Material de la Unidad 1: Representar para construir.

### **13. Aportes individuales**

**Aporte de: Jorge Eliecer Mazo Uparela**

#### **Análisis de requerimientos**

A partir de la contextualización del reto y de la síntesis orientadora de la docente, identifiqué los siguientes requerimientos funcionales que la base de datos debe soportar.

#### **Requerimientos del estudiante**

- La plataforma debe permitir registrar estudiantes del Pascual. Cada estudiante debe contar con un identificador único, nombres, apellidos, correo institucional, fecha de nacimiento, programa académico, semestre y fecha de registro.
- Cada estudiante puede crear su perfil dentro de la plataforma. El perfil se describe con descripción personal, área de estudio, fotografía, biografía y tipo de mentoría.
- Un estudiante puede registrar uno o varios intereses, por ejemplo: Programación, Inteligencia Artificial o Videojuegos. De cada interés se conoce código, nombre y descripción.
- Un estudiante puede registrar una o varias habilidades, por ejemplo: Fútbol, Lectura o Diseño. De cada habilidad se conoce código, nombre y descripción.
- Un estudiante puede seguir a otros estudiantes y ser seguido por ellos, formando una red de conexiones.
- Un estudiante puede publicar actualizaciones: preguntas sobre tareas, recursos útiles, noticias de la industria tecnológica o memes relevantes.
- Un estudiante puede crear grupos de estudio, equipos para hackatones o clubes de interés, y también puede unirse a grupos existentes.
- Un estudiante puede crear eventos, como reuniones de estudio, talleres o actividades sociales, y participar en eventos creados por otros.

### **Requerimientos de las entidades de apoyo**

- La plataforma debe gestionar los programas académicos ofrecidos por la institución. De cada programa se conoce identificador y nombre.

- La plataforma debe gestionar los intereses disponibles, independientemente de los estudiantes que los seleccionen.
- La plataforma debe gestionar las habilidades disponibles, independientemente de los estudiantes que las posean.
- La plataforma debe gestionar los grupos creados, indicando nombre, tipo, descripción y estudiante creador.
- La plataforma debe gestionar los eventos programados, indicando nombre, fecha, hora, lugar, estado y descripción.

### **Requerimientos de integridad**

- Toda publicación debe tener un autor (estudiante) obligatorio.
- Todo grupo debe tener un creador (estudiante) obligatorio.
- Todo evento debe tener un creador (estudiante) obligatorio.
- Todo perfil debe estar asociado a un único estudiante.
- Todo estudiante debe pertenecer a un único programa académico.
- Toda relación de muchos a muchos debe resolverse mediante una entidad intermedia con sus llaves foráneas correspondientes.

### **Entidades identificadas y justificación**

Clasifico las entidades según su naturaleza, siguiendo el material de la unidad:  
personas, objetos, lugares, transacciones y conceptos o abstracciones.

**ESTUDIANTE.** Es de tipo Persona. Es el actor central de la red social. Crea perfil, publica, sigue, crea grupos y participa en eventos. Tiene identidad propia y datos únicos.

**PROGRAMA.** Es de tipo Concepto o abstracción. Los programas académicos se repiten entre estudiantes, por ejemplo, varios estudiantes de Tecnología en Desarrollo de Software o de Ingeniería Eléctrica. Tiene atributos propios: identificador y nombre. No es un atributo de Estudiante porque un programa agrupa a muchos estudiantes y existe independientemente de ellos.

**PERFIL.** Es de tipo Concepto o abstracción. Es la identidad social y personal del estudiante dentro de la red. La docente lo propone como entidad separada. Tiene atributos propios: descripción personal, área de estudio, fotografía, biografía y tipo de mentoría. Se relaciona uno a uno con Estudiante.

**INTERES.** Es de tipo Concepto o abstracción. Los intereses se repiten entre varios estudiantes, por ejemplo, varios con interés en videojuegos, inteligencia artificial o programación. Tiene atributos propios: código, nombre y descripción. No es atributo multivaluado de Estudiante porque debe poder consultarse de forma independiente.

**HABILIDAD.** Es de tipo Concepto o abstracción. Igual que los intereses, las habilidades se repiten entre estudiantes, por ejemplo, fútbol, lectura o diseño. Tiene atributos propios: código, nombre y descripción.

**PUBLICACION.** Es de tipo Transacción. Corresponde a las actualizaciones que el estudiante comparte: preguntas, recursos, noticias o memes. Tiene atributos propios: código, tipo, contenido y fecha de publicación. Ocurre en el tiempo y pertenece a un autor.

**GRUPO.** Es de tipo Objeto o concepto. Agrupa grupos de estudio, equipos de hackatón o clubes de interés. Tiene atributos propios: código, nombre, tipo y descripción. Es creado por un estudiante y tiene varios miembros.

**EVENTO.** Es de tipo Transacción. Corresponde a reuniones de estudio, talleres o actividades sociales que se programan y coordinan. Tiene atributos propios: código, nombre, fecha, hora, lugar, estado y descripción. Es creado por un estudiante y tiene varios participantes.

### **Entidades intermedias del rompimiento del modelo lógico**

Como la profesora pidió aplicar el rompimiento del modelo lógico desde el modelo conceptual, las relaciones de muchos a muchos se convierten en entidades intermedias con sus respectivas llaves foráneas. Estas entidades son las siguientes.

**ESTUDIANTE\_INTERES.** Entidad intermedia que resuelve la relación de muchos a muchos entre ESTUDIANTE e INTERES. Sus atributos son codigo\_estudiante como llave foránea y codigo\_interes como llave foránea. La llave primaria es compuesta: codigo\_estudiante más codigo\_interes.

**ESTUDIANTE\_HABILIDAD.** Entidad intermedia que resuelve la relación de muchos a muchos entre ESTUDIANTE y HABILIDAD. Sus atributos son codigo\_estudiante como llave foránea y codigo\_habilidad como llave foránea. La llave primaria es compuesta: codigo\_estudiante más codigo\_habilidad.

**SEGUIMIENTO.** Entidad intermedia que resuelve la relación reflexiva de muchos a muchos entre ESTUDIANTE y ESTUDIANTE. Sus atributos son codigo\_seguidor como llave foránea que referencia a ESTUDIANTE y codigo\_seguido como llave foránea que referencia a ESTUDIANTE. La llave primaria es compuesta: codigo\_seguidor más codigo\_seguido. Además, se puede incluir el atributo fecha\_seguimiento como dato adicional.

**MIEMBRO\_GRUPO.** Entidad intermedia que resuelve la relación de muchos a muchos entre ESTUDIANTE y GRUPO, correspondiente a la acción de unirse a un grupo. Sus atributos son codigo\_estudiante como llave foránea y codigo\_grupo como llave foránea. La llave primaria es compuesta: codigo\_estudiante más codigo\_grupo. Además, se puede incluir el atributo fecha\_union como dato adicional.

**PARTICIPANTE\_EVENTO.** Entidad intermedia que resuelve la relación de muchos a muchos entre ESTUDIANTE y EVENTO, correspondiente a la acción de participar en un evento. Sus atributos son codigo\_estudiante como llave foránea y codigo\_evento como llave foránea. La llave primaria es compuesta: codigo\_estudiante más codigo\_evento. Además, se puede incluir el atributo fecha\_inscripcion como dato adicional.

### **Atributos de cada entidad con llaves foráneas explícitas**

Convenciones: PK significa clave primaria, FK significa clave foránea, el asterisco indica obligatorio y el guion indica opcional.

#### **ESTUDIANTE**

- codigo\_estudiante: PK, obligatorio. Identificador único del estudiante.
- nombre: texto, obligatorio. Nombres del estudiante.
- apellido: texto, obligatorio. Apellidos del estudiante.
- correo\_institucional: texto, obligatorio. Correo institucional del Pascual.
- fecha\_nacimiento: fecha, obligatorio. Fecha de nacimiento.
- semestre: numérico, obligatorio. Semestre que cursa.
- fecha\_registro: fecha, obligatorio. Fecha en que se registró en la plataforma.
- codigo\_programa: FK, obligatorio. Referencia a PROGRAMA(codigo\_programa).

Justificación: el programa académico se separa como entidad PROGRAMA y se referencia con una llave foránea, porque un programa agrupa a muchos estudiantes y tiene atributos propios. Esta es una relación de muchos a uno, donde la llave foránea va en el lado muchos, es decir, en ESTUDIANTE. El perfil también se separa como entidad PERFIL por decisión de la docente y porque tiene atributos específicos que no pertenecen a la identidad básica del estudiante.

## **PROGRAMA**

- codigo\_programa: PK, obligatorio. Identificador único del programa.
- nombre\_programa: texto, obligatorio. Nombre del programa académico.

Justificación: es una entidad independiente porque los programas existen aunque no haya estudiantes registrados, y varios estudiantes comparten el mismo programa.

## **PERFIL**

- codigo\_perfil: PK, obligatorio. Identificador único del perfil.
- descripcion\_personal: texto, opcional. Descripción personal del estudiante.
- area\_estudio: texto, obligatorio. Área de estudio principal.
- fotografia: texto, opcional. URL o ruta de la fotografía.
- biografia: texto, opcional. Biografía del estudiante.
- tipo\_mentoria: texto, opcional. Tipo de mentoría que ofrece, si aplica.
- codigo\_estudiante: FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).

Justificación: la docente propone Perfil como entidad separada. Se relaciona uno a uno con Estudiante porque cada estudiante tiene un solo perfil y cada perfil pertenece a un solo estudiante. En el rompimiento del modelo lógico, la llave foránea se ubica en PERFIL, que es el lado que depende de ESTUDIANTE.

## **INTERES**

- codigo\_interes: PK, obligatorio. Identificador único del interés.
- nombre\_interes: texto, obligatorio. Nombre del interés.
- descripcion: texto, opcional. Descripción del interés.

Justificación: es una entidad independiente porque los intereses se repiten entre estudiantes y deben poder consultarse y administrarse por separado.

## **HABILIDAD**

- codigo\_habilidad: PK, obligatorio. Identificador único de la habilidad.
- nombre\_habilidad: texto, obligatorio. Nombre de la habilidad.
- descripcion: texto, opcional. Descripción de la habilidad.

Justificación: igual que los intereses, las habilidades se repiten entre estudiantes y tienen atributos propios.

## **PUBLICACION**

- codigo\_publicacion: PK, obligatorio. Identificador único de la publicación.
- tipo\_publicacion: texto, obligatorio. Puede ser pregunta, recurso, noticia o meme.

- contenido: texto, obligatorio. Contenido de la publicación.
- fecha\_publicacion: fecha, obligatorio. Fecha en que se publicó.
- codigo\_estudiante: FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).

Justificación: se corrige el error del foro, donde se habían asignado atributos de Evento a Publicación. Publicación no tiene fecha\_evento, hora\_evento ni lugar; esos son de Evento. En el rompimiento del modelo lógico, la relación uno a muchos entre ESTUDIANTE y PUBLICACION coloca la llave foránea en PUBLICACION, que es el lado muchos.

## GRUPO

- codigo\_grupo: PK, obligatorio. Identificador único del grupo.
- nombre\_grupo: texto, obligatorio. Nombre del grupo.
- tipo\_grupo: texto, obligatorio. Puede ser estudio, hackatón o club de interés.
- descripcion: texto, opcional. Descripción del grupo.
- codigo\_estudiante\_creador: FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).

Justificación: el grupo tiene un creador obligatorio, pero también puede tener varios miembros, lo que se modela con la entidad intermedia MIEMBRO\_GRUPO. En el rompimiento del modelo lógico, la relación uno a muchos entre ESTUDIANTE y GRUPO, correspondiente a la acción de crear, coloca la llave foránea en GRUPO, que es el lado muchos.

## EVENTO

- `codigo_evento`: PK, obligatorio. Identificador único del evento.
- `nombre_evento`: texto, obligatorio. Nombre del evento.
- `fecha_evento`: fecha, obligatorio. Fecha del evento.
- `hora_evento`: hora, obligatorio. Hora del evento.
- `lugar`: texto, obligatorio. Lugar del evento.
- `estado`: texto, obligatorio. Puede ser programado, realizado o cancelado.
- `descripcion`: texto, opcional. Descripción del evento.
- `codigo_estudiante_creador`: FK, obligatorio. Referencia a `ESTUDIANTE(codigo_estudiante)`.

Justificación: el evento tiene un creador obligatorio y varios participantes, lo que se modela con la entidad intermedia `PARTICIPANTE_EVENTO`. En el rompimiento del modelo lógico, la relación uno a muchos entre `ESTUDIANTE` y `EVENTO`, correspondiente a la acción de crear, coloca la llave foránea en `EVENTO`, que es el lado muchos.

### **ESTUDIANTE\_INTERES**

- `codigo_estudiante`: PK, FK, obligatorio. Referencia a `ESTUDIANTE(codigo_estudiante)`.
- `codigo_interes`: PK, FK, obligatorio. Referencia a `INTERES(codigo_interes)`.
- `fecha_asociacion`: fecha, opcional. Fecha en que el estudiante registró el interés.

Justificación: resuelve la relación de muchos a muchos entre `ESTUDIANTE` e `INTERES`. La llave primaria es compuesta por las dos llaves foráneas.

### **ESTUDIANTE\_HABILIDAD**

- `codigo_estudiante`: PK, FK, obligatorio. Referencia a `ESTUDIANTE(codigo_estudiante)`.
- `codigo_habilidad`: PK, FK, obligatorio. Referencia a `HABILIDAD(codigo_habilidad)`.
- `nivel`: texto, opcional. Nivel de dominio de la habilidad.

Justificación: resuelve la relación de muchos a muchos entre `ESTUDIANTE` y

`HABILIDAD`. La llave primaria es compuesta por las dos llaves foráneas.

## **SEGUIMIENTO**

- `codigo_seguidor`: PK, FK, obligatorio. Referencia a `ESTUDIANTE(codigo_estudiante)`.
- `codigo_seguido`: PK, FK, obligatorio. Referencia a `ESTUDIANTE(codigo_estudiante)`.
- `fecha_seguimiento`: fecha, opcional. Fecha en que comenzó el seguimiento.

Justificación: resuelve la relación reflexiva de muchos a muchos entre `ESTUDIANTE` y `ESTUDIANTE`. La llave primaria es compuesta por las dos llaves foráneas, que referencian a la misma entidad pero con roles distintos: seguidor y seguido.

## **MIEMBRO\_GRUPO**

- `codigo_estudiante`: PK, FK, obligatorio. Referencia a `ESTUDIANTE(codigo_estudiante)`.
- `codigo_grupo`: PK, FK, obligatorio. Referencia a `GRUPO(codigo_grupo)`.
- `fecha_union`: fecha, opcional. Fecha en que el estudiante se unió al grupo.

- rol: texto, opcional. Rol del estudiante dentro del grupo.

Justificación: resuelve la relación de muchos a muchos entre ESTUDIANTE y GRUPO, correspondiente a la acción de unirse. La llave primaria es compuesta por las dos llaves foráneas.

## **PARTICIPANTE\_EVENTO**

- codigo\_estudiante: PK, FK, obligatorio. Referencia a ESTUDIANTE(codigo\_estudiante).
- codigo\_evento: PK, FK, obligatorio. Referencia a EVENTO(codigo\_evento).
- fecha\_inscripcion: fecha, opcional. Fecha en que el estudiante se inscribió al evento.
- asistencia: texto, opcional. Indica si asistió o no.

Justificación: resuelve la relación de muchos a muchos entre ESTUDIANTE y EVENTO, correspondiente a la acción de participar. La llave primaria es compuesta por las dos llaves foráneas.

## **Relaciones, cardinalidades y participación con rompimiento del modelo lógico**

Convenciones: uno a uno, uno a muchos, muchos a muchos. Debe indica participación obligatoria; puede indica participación opcional.

Relación tiene entre ESTUDIANTE y PERFIL. Cardinalidad uno a uno. Participación: Estudiante debe, Perfil debe. Rompimiento: la llave foránea codigo\_estudiante se ubica en PERFIL. Justificación: todo estudiante debe tener un perfil; todo perfil pertenece a un estudiante.

Relación pertenece entre ESTUDIANTE y PROGRAMA. Cardinalidad muchos a uno.

Participación: Estudiante debe, Programa puede. Rompimiento: la llave foránea

codigo\_programa se ubica en ESTUDIANTE. Justificación: todo estudiante debe pertenecer a un programa; un programa puede tener muchos estudiantes.

Relación tiene entre ESTUDIANTE e INTERES. Cardinalidad muchos a muchos.

Participación: Estudiante puede, Interés puede. Rompimiento: se crea la entidad intermedia

ESTUDIANTE\_INTERES con las llaves foráneas codigo\_estudiante y codigo\_interes.

Justificación: un estudiante puede tener cero o varios intereses; un interés puede estar asociado a cero o varios estudiantes.

Relación posee entre ESTUDIANTE y HABILIDAD. Cardinalidad muchos a muchos.

Participación: Estudiante puede, Habilidad puede. Rompimiento: se crea la entidad intermedia

ESTUDIANTE\_HABILIDAD con las llaves foráneas codigo\_estudiante y codigo\_habilidad.

Justificación: un estudiante puede poseer cero o varias habilidades; una habilidad puede estar asociada a cero o varios estudiantes.

Relación sigue entre ESTUDIANTE y ESTUDIANTE. Cardinalidad muchos a muchos.

Participación: Estudiante puede, Estudiante puede. Rompimiento: se crea la entidad intermedia

SEGUIMIENTO con las llaves foráneas codigo\_seguidor y codigo\_seguido, ambas

referenciando a ESTUDIANTE. Justificación: es una relación reflexiva. Un estudiante puede seguir a cero o varios estudiantes y ser seguido por cero o varios.

Relación publica entre ESTUDIANTE y PUBLICACION. Cardinalidad uno a muchos.

Participación: Estudiante puede, Publicación debe. Rompimiento: la llave foránea

codigo\_estudiante se ubica en PUBLICACION. Justificación: un estudiante puede tener cero o varias publicaciones; toda publicación debe tener un autor.

Relación crea entre ESTUDIANTE y GRUPO. Cardinalidad uno a muchos.

Participación: Estudiante puede, Grupo debe. Rompimiento: la llave foránea

codigo\_estudiante\_creador se ubica en GRUPO. Justificación: un estudiante puede crear cero o varios grupos; todo grupo debe tener un creador.

Relación se une entre ESTUDIANTE y GRUPO. Cardinalidad muchos a muchos.

Participación: Estudiante puede, Grupo puede. Rompimiento: se crea la entidad intermedia

MIEMBRO\_GRUPO con las llaves foráneas codigo\_estudiante y codigo\_grupo. Justificación: un estudiante puede unirse a cero o varios grupos; un grupo puede tener cero o varios miembros.

Relación crea entre ESTUDIANTE y EVENTO. Cardinalidad uno a muchos.

Participación: Estudiante puede, Evento debe. Rompimiento: la llave foránea

codigo\_estudiante\_creador se ubica en EVENTO. Justificación: un estudiante puede crear cero o varios eventos; todo evento debe tener un creador.

Relación participa entre ESTUDIANTE y EVENTO. Cardinalidad muchos a muchos.

Participación: Estudiante puede, Evento puede. Rompimiento: se crea la entidad intermedia

PARTICIPANTE\_EVENTO con las llaves foráneas codigo\_estudiante y codigo\_evento.

Justificación: un estudiante puede participar en cero o varios eventos; un evento puede tener cero o varios participantes.

La relación sigue es reflexiva porque conecta ESTUDIANTE consigo misma. Se debe representar con dos roles: seguidor y seguido. En el rompimiento del modelo lógico, esto se traduce en la entidad SEGUIMIENTO con dos llaves foráneas que referencian a la misma entidad ESTUDIANTE.

### **Posibles jerarquías o generalizaciones**

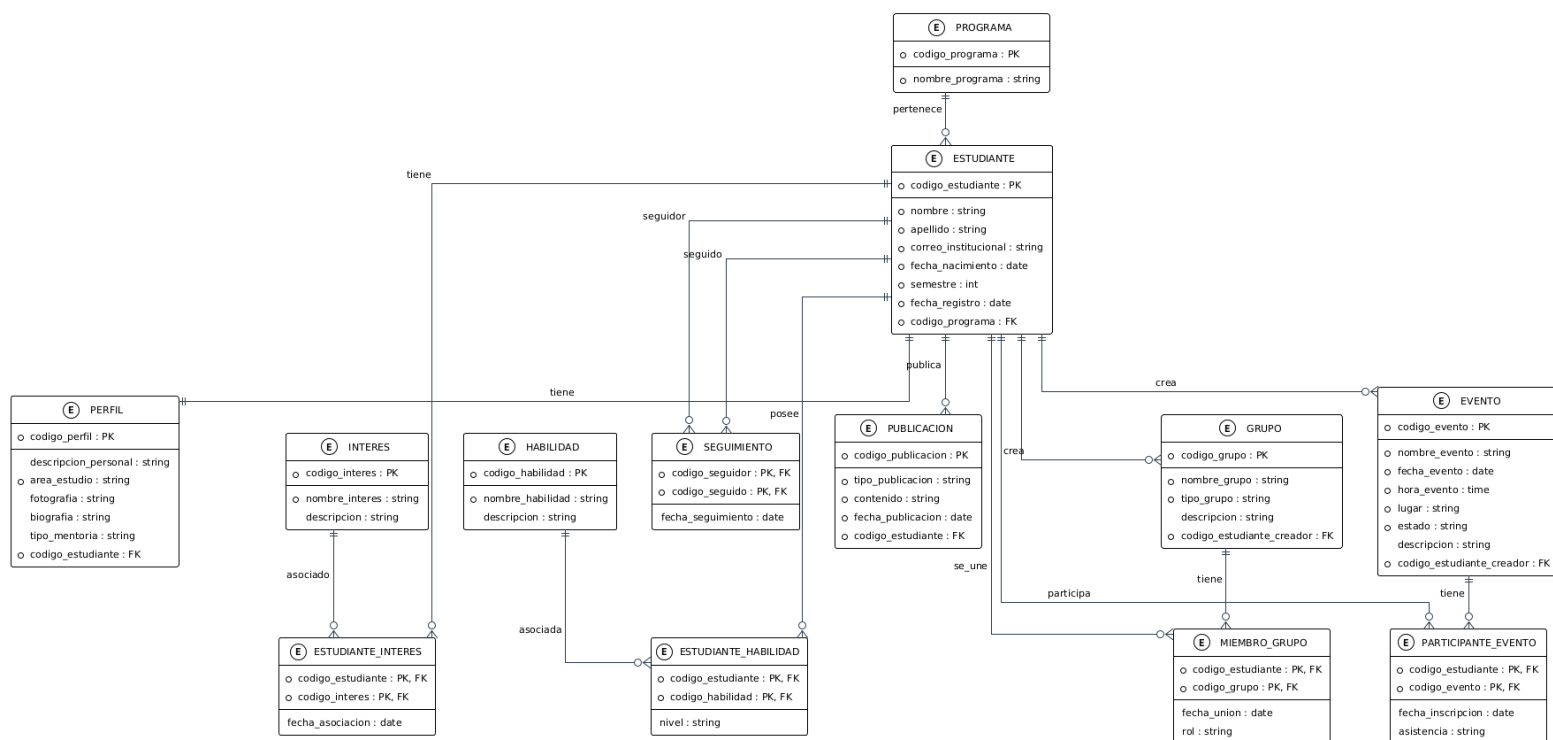
Para PUBLICACION, podría tener subtipos Pregunta, Recurso, Noticia y Meme.

Decisión: no se modela como jerarquía porque el tipo se puede representar con el atributo tipo\_publicacion. Si el grupo quiere mayor detalle, podría hacerse, pero no es obligatorio.

Para EVENTO, podría tener subtipos Estudio, Taller y Social. Decisión: no se modela como jerarquía por la misma razón: el atributo tipo\_evento, si se agrega, sería suficiente.

Para GRUPO, podría tener subtipos Estudio, Hackatón y Club. Decisión: no se modela como jerarquía; el atributo tipo\_grupo cubre la necesidad.

### Diagrama MER borrador individual



**Aporte de: Samuel Hincapié Pérez**

### **Requerimientos funcionales que sustentan el modelo**

#### **RF1. Registro de estudiantes**

El sistema debe permitir registrar estudiantes con sus datos básicos, como código, nombre, correo institucional, estado y fecha de registro.

#### **RF2. Asociación con un programa académico**

Cada estudiante debe estar relacionado con un programa académico, mientras que un programa puede tener varios estudiantes registrados.

#### **RF3. Registro de habilidades**

El sistema debe permitir que un estudiante tenga varias habilidades y que una misma habilidad pueda ser compartida por diferentes estudiantes.

#### **RF4. Registro de intereses**

El sistema debe permitir que los estudiantes tengan varios intereses y que un mismo interés pueda estar relacionado con diferentes estudiantes.

#### **RF5. Seguimiento entre estudiantes**

El sistema debe permitir que los estudiantes sigan a otros estudiantes y que también puedan ser seguidos por varias personas.

#### **RF6. Publicación de contenido**

El sistema debe permitir que los estudiantes publiquen contenido. Cada publicación debe estar relacionada con el estudiante que la realizó.

#### **RF7. Creación de grupos**

El sistema debe permitir que un estudiante cree un grupo, teniendo cada grupo un único creador.

#### RF8. Unión a grupos

El sistema debe permitir que los estudiantes se unan a diferentes grupos, sin importar quién haya creado cada grupo.

#### RF9. Creación de eventos

El sistema debe permitir que un estudiante cree eventos, teniendo cada evento un único creador.

#### RF10. Participación en eventos

El sistema debe permitir que los estudiantes se inscriban o participen en diferentes eventos.

#### RF11. Organización de eventos

El sistema debe permitir que un grupo organice eventos, de manera que un grupo pueda organizar varios eventos.

### Análisis de necesidades por entidad

#### Estudiante

La entidad Estudiante es la principal del modelo, porque prácticamente todas las actividades del sistema están relacionadas con ella. Aquí se guardan los datos básicos de cada estudiante, como su nombre, correo institucional, estado y fecha de registro.

También se relaciona con programas, habilidades, intereses, publicaciones, grupos, eventos y otros estudiantes. Por eso se puede decir que es el centro del modelo, ya que las demás entidades representan actividades o información relacionada con los estudiantes.

### Programa

La entidad Programa se utiliza para almacenar la información de los programas académicos, como su código, nombre y facultad.

Se decidió manejarla como una tabla independiente para no repetir estos datos en cada estudiante. Por ejemplo, si varios estudiantes pertenecen al mismo programa, no sería necesario escribir nuevamente el nombre del programa y la facultad en cada registro.

Esto ayuda a evitar la repetición de información y facilita realizar cambios cuando sea necesario.

### Habilidad

La entidad Habilidad representa las diferentes capacidades que pueden tener los estudiantes, como programación, liderazgo, diseño, entre otras.

Se encuentra separada de Estudiante porque una misma habilidad puede pertenecer a muchos estudiantes. Por ejemplo, varios estudiantes pueden tener la habilidad de programación.

Si las habilidades se guardaran directamente en Estudiante, sería más complicado consultar cuáles estudiantes tienen una determinada habilidad.

### Interes

La entidad Interes representa los temas o actividades que les gustan a los estudiantes, como música, videojuegos, fotografía o tecnología.

Se maneja de forma independiente porque un mismo interés puede estar relacionado con varios estudiantes. Además, se separa de Habilidad porque no es lo mismo tener conocimiento o capacidad en algo que simplemente tener interés en ese tema.

### Publicacion

La entidad Publicacion representa el contenido que los estudiantes comparten en la plataforma.

Se creó como una tabla independiente porque un estudiante puede realizar muchas publicaciones y cada una necesita guardar información propia, como el tipo de publicación, el contenido y la fecha en que fue realizada.

También tiene una clave foránea que permite saber qué estudiante hizo cada publicación.

### Grupo

La entidad Grupo representa comunidades o espacios de integración dentro de la plataforma. Por ejemplo, podría ser un grupo de estudio, un club o un semillero.

La tabla guarda información como el nombre, tipo y fecha de creación del grupo. Además, tiene el campo `codigo_estudiante`, que permite identificar al estudiante que creó ese grupo.

Aquí es importante diferenciar entre el estudiante que crea el grupo y los estudiantes que se unen a él, porque no necesariamente son la misma persona.

### Evento

La entidad Evento representa actividades que se realizan dentro de la plataforma, como charlas, talleres, reuniones o actividades académicas.

Tiene información como nombre, hora y lugar. Además, en el modelo aparecen dos claves foráneas relacionadas con estudiantes y grupos:

codigo\_estudiante: identifica al estudiante que creó el evento.

codigo\_grupo: identifica al grupo que organiza el evento.

Esto permite diferenciar quién creó el evento y qué grupo está encargado de organizarlo.

Análisis de relaciones: cardinalidad y justificación

Para decidir cómo representar cada relación, se analizó cuántos registros de una entidad pueden relacionarse con la otra.

La regla utilizada fue la siguiente:

Si ambos lados pueden tener varios registros relacionados, se utiliza una tabla puente.

Si un lado tiene un único registro relacionado y el otro puede tener varios, se utiliza una clave foránea directa.

Relaciones muchos a muchos

Relación posee: Estudiante - Habilidad

La relación posee es de muchos a muchos porque un estudiante puede tener varias habilidades y una misma habilidad puede pertenecer a varios estudiantes.

Por ejemplo, un estudiante puede tener conocimientos en programación, diseño y bases de datos, mientras que otros estudiantes también pueden tener esas mismas habilidades.

Por esta razón se creó la tabla puente Estudiante\_Habilidad, que contiene:

codigo\_estudiante

codigo\_habilidad

Esta tabla permite registrar qué habilidades tiene cada estudiante sin repetir información.

Relación tiene: Estudiante - Interes

La relación tiene también es muchos a muchos.

Un estudiante puede tener varios intereses y un mismo interés puede estar asociado a diferentes estudiantes.

Por ejemplo, varios estudiantes pueden tener interés en los videojuegos, mientras que uno de ellos también puede tener interés en música y tecnología.

Para resolver esta relación se utiliza la tabla puente Estudiante\_Interes, que contiene:

codigo\_estudiante

codigo\_interes

De esta forma se pueden registrar todas las combinaciones entre estudiantes e intereses.

Relación sigue: Estudiante - Estudiante

La relación sigue es un caso especial porque Estudiante se relaciona consigo misma.

Esto significa que un estudiante puede seguir a muchos otros y también puede ser seguido por muchos estudiantes.

Por ejemplo:

El estudiante 101 sigue al estudiante 205.

Para representar esta relación se creó la tabla Seguimiento, que contiene:

codigo\_estudiante\_seguidor

codigo\_estudiante\_seguido

fecha\_seguimiento

Aunque ambos códigos vienen de la tabla Estudiante, cumplen funciones diferentes:

El seguidor es quien realiza el seguimiento.

El seguido es la persona que recibe ese seguimiento.

Por eso se necesitan dos claves foráneas hacia la misma tabla.

Relación se une a: Estudiante - Grupo

La relación se une a es muchos a muchos porque un estudiante puede pertenecer a varios grupos y un grupo puede tener muchos estudiantes.

Por ejemplo, un estudiante puede estar en un grupo de programación y también en uno de investigación.

Para resolver esta relación se utiliza la tabla Estudiante\_Grupo, que contiene:

codigo\_estudiante

codigo\_grupo

fecha\_union

La fecha de unión es importante porque permite saber cuándo un estudiante ingresó a un grupo específico.

Relación se une a: Estudiante - Evento

La relación entre Estudiante y Evento también es muchos a muchos, ya que un estudiante puede participar en varios eventos y un evento puede tener varios participantes.

Para resolverla se utiliza la tabla Estudiante\_Evento, que contiene:

codigo\_estudiante

codigo\_evento

fecha\_inscripcion

Esta tabla permite registrar qué estudiantes participan en cada evento y cuándo se inscribieron.

En estos casos se utilizan tablas puente porque no sería suficiente colocar una sola clave foránea en una de las tablas principales. Eso limitaría la cantidad de relaciones que se pueden registrar y se perdería información.

Relaciones uno a muchos

Relación pertenece a: Programa - Estudiante

Esta relación es uno a muchos porque un programa puede tener varios estudiantes, pero cada estudiante pertenece a un programa académico.

Por eso la clave foránea `codigo_programa` se encuentra en la tabla Estudiante.

No se necesita una tabla puente porque cada estudiante solo está asociado a un programa dentro del modelo.

Relación publica: Estudiante - Publicacion

Esta relación es uno a muchos porque un estudiante puede realizar varias publicaciones, pero cada publicación pertenece a un solo estudiante.

Por eso la tabla Publicacion contiene la clave foránea `codigo_estudiante`, que permite identificar al autor de cada publicación.

Relación crea: Estudiante - Grupo

La relación crea es uno a muchos porque un estudiante puede crear varios grupos, pero cada grupo tiene un único creador.

En el diagrama, esta relación se representa mediante el campo:

`codigo_estudiante` en la tabla Grupo.

Este campo permite saber qué estudiante creó cada grupo.

Es importante no confundir esta relación con se une a, porque crear un grupo y pertenecer a él son cosas diferentes.

Un estudiante puede crear un grupo y luego otros estudiantes pueden unirse mediante la tabla Estudiante\_Grupo.

Relación crea: Estudiante - Evento

La relación crea entre Estudiante y Evento también es uno a muchos.

Un estudiante puede crear varios eventos, pero cada evento tiene un único creador.

Por eso la tabla Evento contiene el campo:

codigo\_estudiante

Este campo identifica al estudiante que creó el evento.

Sin embargo, esto no significa que ese estudiante sea el único que participe en el evento, ya que los demás participantes se registran en la tabla Estudiante\_Evento.

Relación organiza: Grupo - Evento

La relación organiza es uno a muchos porque un grupo puede organizar varios eventos, mientras que cada evento está asociado a un grupo organizador.

Por eso la tabla Evento contiene la clave foránea:

codigo\_grupo

Esta permite identificar qué grupo organiza cada evento.

Esta relación es diferente a la de crear, porque un estudiante puede crear el evento, pero un grupo puede ser el encargado de organizarlo.

Diferencia entre las relaciones parecidas

En el modelo hay algunas relaciones que pueden confundirse, por eso es importante diferenciarlas.

Crear un grupo

Indica quién fue el estudiante que creó el grupo.

Se representa con:

Grupo → codigo\_estudiante

Unirse a un grupo

Indica qué estudiantes pertenecen a ese grupo.

Se representa con:

Estudiante\_Grupo

Crear un evento

Indica qué estudiante creó el evento.

Se representa con:

Evento → codigo\_estudiante

Participar en un evento

Indica qué estudiantes están inscritos o participan en el evento.

Se representa con:

Estudiante\_Evento

Organizar un evento

Indica qué grupo está encargado de organizarlo.

Se representa con:

Evento → codigo\_grupo

Aunque algunas relaciones parecen similares, cada una representa una acción diferente dentro del sistema.

### **Conclusión**

En conclusión, mi modelo se diseñó teniendo en cuenta las necesidades principales del sistema y la forma en que se relacionan sus entidades.

Las relaciones muchos a muchos se resolvieron mediante tablas puente, como:

Estudiante\_Habilidad.

Estudiante\_Interes.

Seguimiento.

Estudiante\_grupo.

Estudiante\_Evento.

Por otro lado, las relaciones uno a muchos se resolvieron utilizando claves foráneas directas, como:

Estudiante con Programa.

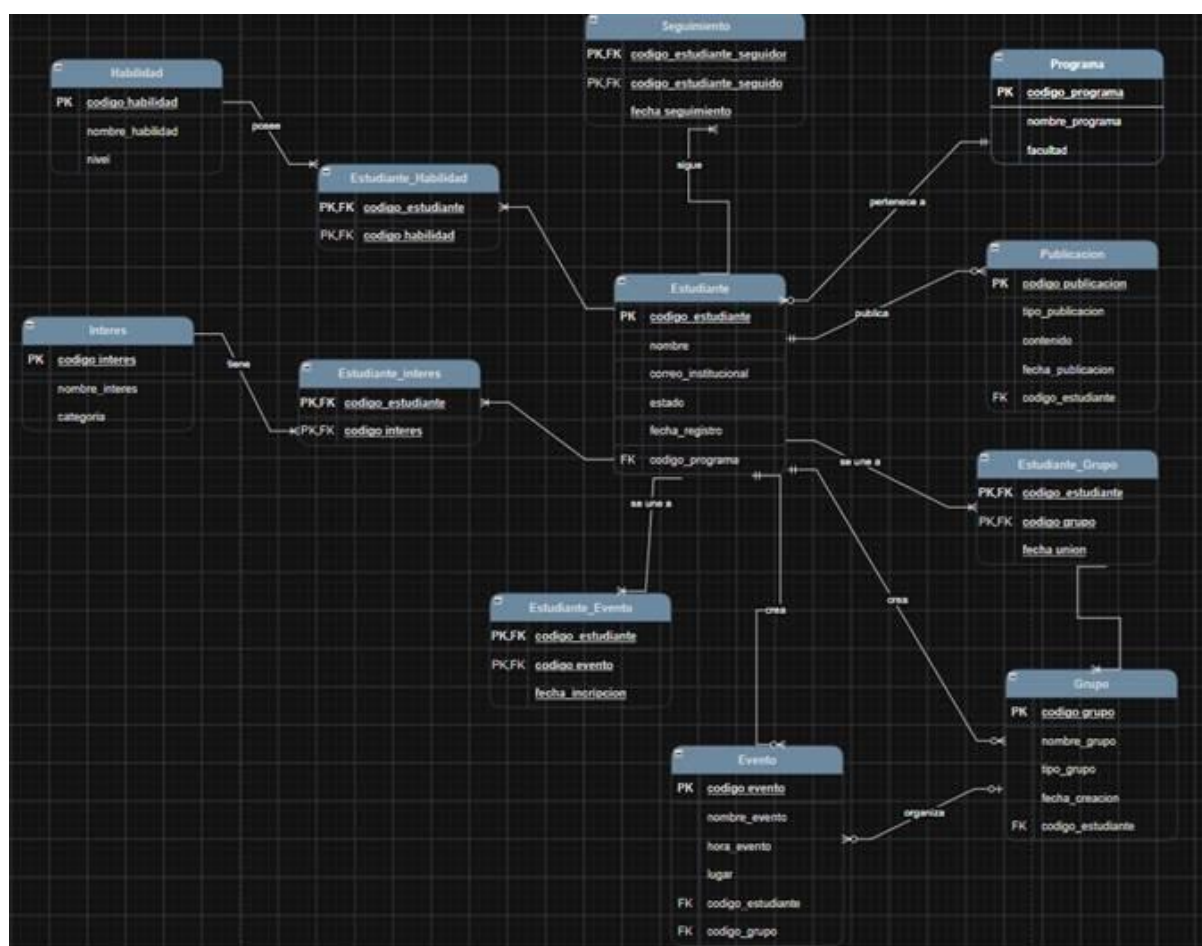
Publicacion con Estudiante.

Grupo con Estudiante.

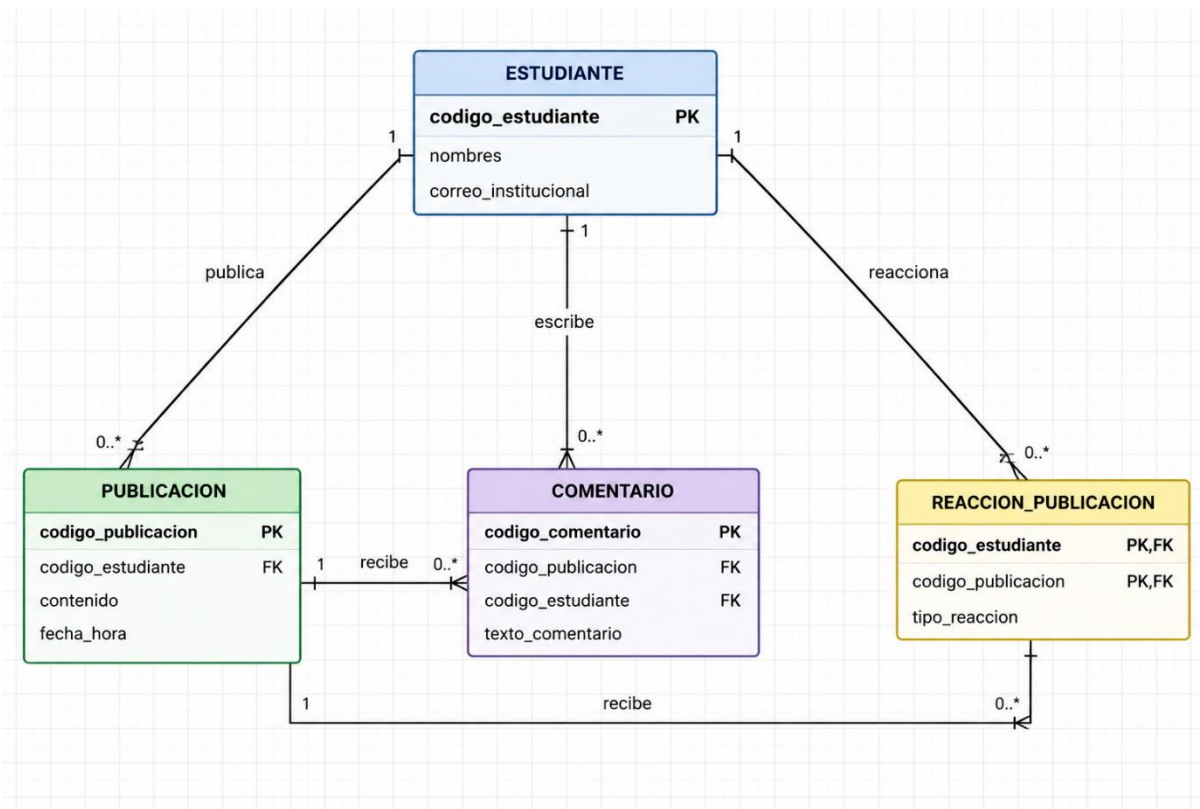
Evento con Estudiante.

Evento con Grupo.

De esta manera, el modelo permite representar correctamente la información de los estudiantes, sus habilidades, intereses, publicaciones, grupos y eventos, evitando repetición de datos.



### Aporte de: Karen Castro Arias



### Aporte de: Ali Jose Arias Basto

En este aporte documento la parte del modelo que me correspondió: el módulo de recursos académicos, que permite a los estudiantes compartir material de estudio dentro de la red y valorarlo. Este módulo lo conforman tres entidades: `RECURSO_ACADEMICO`, `TIPO_RECURSO` y `VALORACION_RECURSO`.

Requerimientos funcionales que sustentan mi parte

RF1. El sistema debe permitir que un estudiante comparta recursos académicos (apuntes, enlaces, código, videos o documentos), guardando el título, el archivo o enlace y la fecha en que se publicó.

RF2. Cada recurso debe estar asociado a la materia a la que pertenece, para que otros estudiantes puedan encontrarlo por asignatura.

RF3. Cada recurso debe estar clasificado por un tipo (por ejemplo: PDF, enlace, video, código fuente), de manera que el mismo tipo pueda usarse en muchos recursos distintos.

RF4. El sistema debe permitir que los estudiantes valoren los recursos con una puntuación y un comentario, de forma que un estudiante pueda valorar muchos recursos y un recurso pueda recibir valoraciones de muchos estudiantes.

#### Análisis de necesidades por entidad

##### RECURSO\_ACADEMICO

Es la entidad central de mi módulo. Representa el material que un estudiante comparte con sus compañeros. Guarda información propia de cada recurso, como el título, el enlace o archivo y la fecha de publicación.

Se maneja como una tabla independiente porque un estudiante puede compartir muchos recursos, y cada recurso necesita guardar sus propios datos. Además tiene tres claves foráneas que permiten saber quién lo compartió (codigo\_estudiante), a qué materia pertenece (codigo\_materia) y de qué tipo es (codigo\_tipo\_recurso).

##### TIPO\_RECORSO

Es una tabla de catálogo. Clasifica los recursos en categorías como PDF, enlace, video o código fuente. Se separó en su propia tabla para no repetir el nombre del tipo en cada

recurso: en vez de escribir «PDF» una y otra vez, se guarda una sola vez y los recursos lo referencian con una clave foránea.

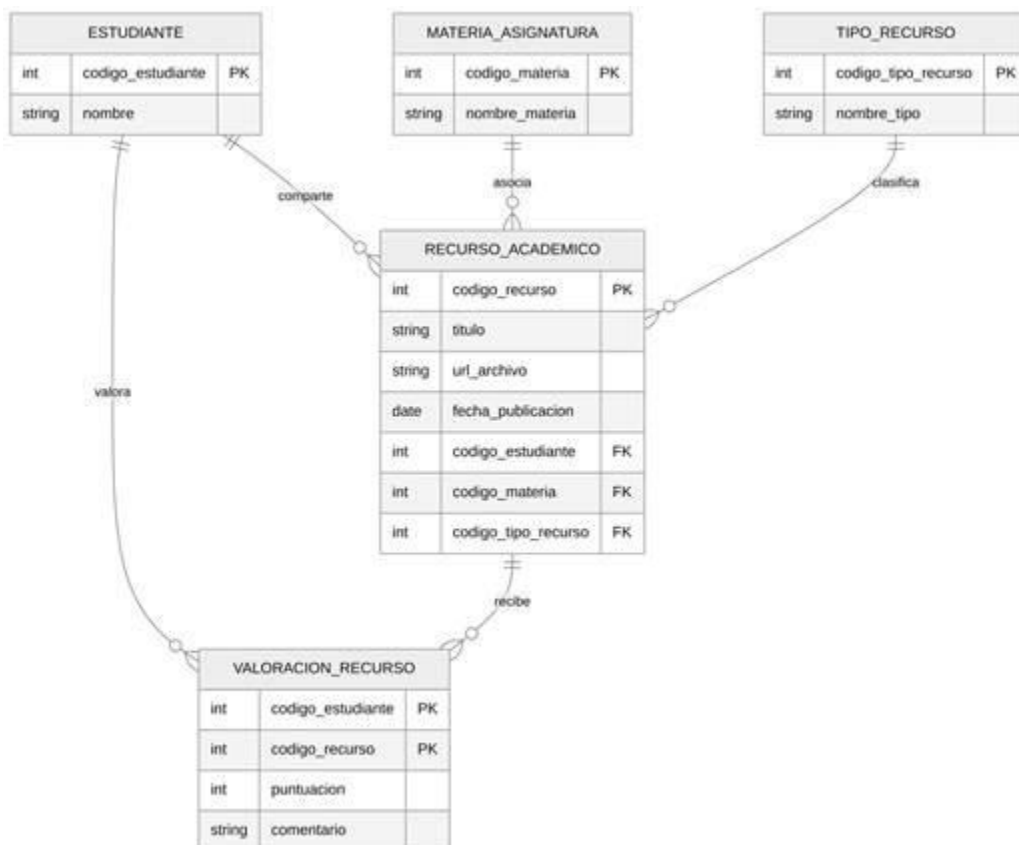
Esto evita repetir información y facilita agregar o cambiar tipos sin tener que modificar cada recurso.

#### VALORACION\_RECURSO

Representa la puntuación y el comentario que un estudiante deja sobre un recurso. Se guarda como una tabla aparte porque la valoración no es un dato ni del estudiante solo ni del recurso solo: es un dato que nace de la combinación de ambos. Guarda la puntuación y el comentario, junto con las referencias al estudiante que valora y al recurso valorado.

#### Diagrama del módulo

La siguiente figura muestra cómo se relacionan las entidades de mi parte con el Estudiante y la Materia. Se usa la notación pata de cuervo, donde la línea con los tres trazos indica el lado «muchos» de la relación.



### Análisis de relaciones: cardinalidad y justificación

Para decidir cómo representar cada relación usé la misma regla del modelo: si un lado tiene un solo registro relacionado y el otro puede tener varios, se usa una clave foránea directa; si ambos lados pueden tener varios, se usa una tabla puente.

Relaciones uno a muchos (se resuelven con clave foránea)

**Comparte:** Estudiante → Recurso. Un estudiante puede compartir muchos recursos, pero cada recurso lo comparte un solo estudiante. Por eso la clave foránea codigo\_estudiante va dentro de RECURSO\_ACADEMICO.

**Asocia:** Materia → Recurso. Una materia puede tener muchos recursos, pero cada recurso pertenece a una sola materia. Por eso codigo\_materia va dentro de RECURSO\_ACADEMICO.

Clasifica: Tipo → Recurso. Un tipo puede aplicarse a muchos recursos, pero cada recurso es de un solo tipo. Por eso `codigo_tipo_recurso` va dentro de `RECURSO_ACADEMICO`. En estos tres casos no hace falta tabla puente porque el lado del recurso siempre apunta a un único estudiante, una única materia y un único tipo.

Relación muchos a muchos (se resuelve con tabla puente)

Valora: Estudiante ↔ Recurso. Un estudiante puede valorar muchos recursos y un recurso puede recibir valoraciones de muchos estudiantes. Como ambos lados son «muchos», no basta una clave foránea: se necesita la tabla puente `VALORACION_RECURSO`, con la llave primaria compuesta por `codigo_estudiante` y `codigo_recurso`. Esa llave compuesta también evita que un mismo estudiante valore dos veces el mismo recurso.

### Conclusión

El módulo de recursos académicos permite que los estudiantes compartan material, lo organicen por materia y por tipo, y lo valoren entre ellos. Las relaciones uno a muchos (compartir, asociar y clasificar) se resolvieron con claves foráneas dentro de `RECURSO_ACADEMICO`, mientras que la relación muchos a muchos de valoración se resolvió con la tabla puente `VALORACION_RECURSO`. De esta forma el módulo representa la información sin repetir datos y encaja con el resto del modelo a través del Estudiante y la Materia.

**Aporte de: Paula Andrea Morales**

## Análisis del modelo entidad–relación

## 1. Identificación de las entidades

El modelo entidad–relación de la Red Social Pascualina permite representar de manera organizada

los principales elementos que hacen parte de la plataforma y las interacciones que pueden realizar

los estudiantes. La entidad principal es ESTUDIANTE, porque representa a los usuarios que utilizan la red social. Cada estudiante posee información propia y puede realizar diferentes acciones dentro de la

plataforma, como seguir a otros estudiantes, crear contenido, pertenecer a grupos, participar en

eventos y registrar sus habilidades e intereses. La entidad HABILIDAD permite almacenar las diferentes capacidades que pueden tener los

estudiantes. Es importante separarla de ESTUDIANTE porque una misma habilidad puede ser

compartida por varios estudiantes. La entidad INTERÉS representa los temas, actividades o áreas que son de interés para los

estudiantes. Esta entidad permite conocer las preferencias de los usuarios y establecer relaciones

entre diferentes estudiantes que pueden tener intereses similares. La entidad GRUPO representa espacios de interacción en los cuales pueden reunirse varios

estudiantes para compartir información, desarrollar actividades o trabajar sobre determinados

temas. La entidad EVENTO representa actividades programadas dentro de la plataforma en las cuales

pueden participar los estudiantes. Estas entidades permiten representar de forma más cercana el funcionamiento de una red social

académica, evitando concentrar toda la información en una sola estructura. 2. Análisis de los atributos

Cada entidad debe contar con atributos que permitan identificarla y describir la información que

se necesita almacenar.

## ESTUDIANTE

□ Clave primaria (PK): id\_estudiante. □ Atributos: nombre, apellido, correo y demás datos definidos en el modelo. □ Función: identificar de manera única a cada estudiante y almacenar la información básica

necesaria del usuario. El atributo `id_estudiante` es importante porque dos estudiantes podrían tener nombres iguales, pero no deberían tener el mismo identificador dentro de la base de datos. HABILIDAD

- Clave primaria (PK): `id_habilidad`.
- Atributo descriptivo: nombre o descripción de la habilidad.
- Función: identificar cada habilidad y almacenar su información. El identificador permite diferenciar las habilidades y posteriormente relacionarlas con los

estudiantes que las poseen. INTERÉS

- Clave primaria (PK): `id_interes`.
- Atributo descriptivo: nombre o descripción del interés.
- Función: identificar cada interés registrado dentro de la plataforma. Esto permite que un mismo interés pueda asociarse con diferentes estudiantes sin tener que

repetir toda la información del interés. GRUPO

- Clave primaria (PK): `id_grupo`.
- Atributos: nombre, descripción y demás información establecida en el modelo.
- Función: identificar cada grupo y almacenar sus características. La información del grupo permite diferenciarlo de otros grupos existentes en la plataforma.

EVENTO

- Clave primaria (PK): `id_evento`.
- Atributos: nombre, fecha, descripción y demás datos definidos.
- Función: identificar cada evento y almacenar la información necesaria para su

organización. Los atributos de fecha y descripción, por ejemplo, permiten conocer cuándo se realiza el evento y

cuál es su finalidad. 3. Análisis de las relaciones

Las relaciones permiten representar las acciones y conexiones que existen entre las diferentes

entidades del sistema. RELACIÓN SIGUE

La relación SIGUE representa la conexión entre estudiantes dentro de la red social. Un estudiante

puede seguir a varios estudiantes y, al mismo tiempo, puede ser seguido por varios compañeros. Esta relación es diferente de las relaciones con grupos o eventos porque representa directamente

una conexión social entre dos usuarios. Además, es una relación particular porque ESTUDIANTE se relaciona consigo misma. En otras

palabras, un estudiante sigue a otro estudiante, por lo que la misma entidad aparece en los dos

extremos de la relación. RELACIÓN CREA

La relación CREA permite identificar al estudiante responsable de crear determinado elemento

dentro de la plataforma, de acuerdo con el diseño establecido en el modelo. Esta relación es importante porque permite diferenciar al usuario que crea un elemento de los

usuarios que posteriormente pueden participar o interactuar con él. Por ejemplo, si un estudiante crea un grupo o un evento, el sistema debe poder identificar quién

fue el creador. Por esta razón, CREA representa una acción diferente de PARTICIPA.

#### RELACIÓN PARTICIPA

La relación PARTICIPA representa la participación de un estudiante en un evento o actividad. Un estudiante puede participar en diferentes eventos y, a su vez, un evento puede contar con la

participación de varios estudiantes. Por esta razón, normalmente corresponde a una relación

muchos a muchos (N:M). Esta relación permite almacenar la interacción entre los usuarios y las actividades que se realizan

dentro de la plataforma. RELACIÓN PERTENECE

La relación PERTENECE representa la relación entre los estudiantes y los grupos. Un estudiante puede pertenecer a diferentes grupos y cada grupo puede estar conformado por

varios estudiantes. Por esta razón, la relación corresponde a una relación N: M. Esta relación es necesaria porque pertenecer a un grupo no significa necesariamente que el

estudiante lo haya creado ni que haya participado en un evento organizado por ese grupo. RELACIÓN CON HABILIDAD

La relación entre ESTUDIANTE y HABILIDAD permite registrar las habilidades que posee cada

estudiante.

Un estudiante puede tener varias habilidades y una misma habilidad puede pertenecer a varios

estudiantes. Por esta razón, la relación es N: M. Esto evita duplicar la información de una habilidad cada vez que otro estudiante la registra. RELACIÓN CON INTERÉS

La relación entre ESTUDIANTE e INTERÉS permite registrar los intereses de cada usuario. Un estudiante puede tener varios intereses y un mismo interés puede estar asociado con

diferentes estudiantes. Por lo tanto, también se trata de una relación N: M. 4. Análisis de las cardinalidades

Las cardinalidades indican cuántas veces puede participar una entidad en una relación. En el modelo se pueden encontrar principalmente relaciones 1: N y N:M, dependiendo de la acción

que se esté representando. 1: N (uno a muchos): significa que un registro de una entidad puede relacionarse con varios

registros de otra entidad. Por ejemplo, un estudiante puede ser responsable de crear varios

elementos, mientras que cada uno de esos elementos tiene un creador determinado.

N:M (muchos a muchos): significa que varios registros de una entidad pueden relacionarse con

varios registros de otra. Este tipo de cardinalidad aparece de manera natural en situaciones como

estudiantes-habilidades, estudiantes-intereses, estudiantes-grupos y estudiantes-eventos. La cardinalidad N:M es especialmente importante en este modelo porque las actividades de una

red social normalmente permiten que muchos usuarios interactúen con diferentes elementos. 5. Flechas y relaciones dobles

Las flechas o líneas que aparecen en el diagrama sirven para representar gráficamente la conexión

entre las entidades y las relaciones. En el caso de una relación como SIGUE, se requiere especial atención porque un estudiante puede

seguir a otro estudiante y también puede ser seguido por otros. Por eso, aunque la relación parte

de la misma entidad ESTUDIANTE, representa dos papeles diferentes dentro de la relación: el

estudiante que sigue y el estudiante que es seguido. Las relaciones dobles no significan necesariamente que existan dos relaciones diferentes. Pueden

representar los dos sentidos o roles que participan en una misma relación. Por ejemplo:

ESTUDIANTE → SIGUE → ESTUDIANTE

permite interpretar que un estudiante establece una conexión con otro estudiante. En el caso de CREA y PARTICIPA, es importante no confundir sus funciones. Un estudiante puede

ser quien crea un evento, pero otros estudiantes pueden participar en ese mismo evento. Por lo

tanto, son acciones diferentes y deben representarse mediante relaciones diferentes.

## 6. Integridad del modelo

La integridad del modelo consiste en comprobar que las entidades, atributos y relaciones tengan

sentido y representen correctamente las necesidades del sistema. En este modelo, cada entidad debe tener una clave primaria (PK) que permita identificarla de

forma única. Además, las relaciones deben tener cardinalidades coherentes con las acciones que

pueden realizar los estudiantes. También es importante evitar almacenar información repetida. Por ejemplo, las habilidades e

intereses deben manejarse como entidades independientes cuando pueden ser compartidos por

varios estudiantes. Otro aspecto importante es evitar relaciones innecesarias. Cada relación debe representar una

acción o asociación real dentro de la plataforma. Esto permite que el modelo sea más claro y

posteriormente pueda convertirse en una base de datos organizada. La integridad también implica que no se deben confundir acciones diferentes. Crear, seguir, pertenecer y participar tienen significados distintos, por lo que cada una debe representar una

situación específica dentro del sistema. 7. Funcionamiento general del modelo

De manera general, el modelo representa el funcionamiento de la Red Social Pascualina de la

siguiente manera:

Un estudiante se registra en la plataforma y cuenta con información propia, además de poder

registrar sus habilidades e intereses. Posteriormente, puede seguir a otros estudiantes y establecer

conexiones dentro de la red social. Los estudiantes también pueden crear o integrarse a grupos. Dentro de estos grupos pueden

desarrollarse diferentes actividades y organizarse eventos. Cuando se crea un evento, se puede identificar quién fue el estudiante que lo creó. Otros

estudiantes pueden participar en dicho evento, generando una relación entre los estudiantes y las

actividades disponibles. De esta manera, el modelo permite representar diferentes niveles de interacción: las relaciones

sociales entre estudiantes, las características personales de los usuarios, la pertenencia a grupos y

la participación en eventos. El modelo entidad–relación sirve entonces como una representación previa de la estructura que

posteriormente puede convertirse en tablas dentro de una base de datos. Cada entidad puede

convertirse en una tabla, sus atributos en campos y las relaciones pueden representarse mediante

claves primarias y claves foráneas o tablas intermedias cuando se presentan relaciones

N: M. Conclusión del análisis

El análisis del modelo entidad–relación permitió comprender que el diseño de una base de datos

requiere estudiar primero las necesidades del sistema y las diferentes interacciones que ocurren

entre sus usuarios.

En la Red Social Pascualina, la entidad ESTUDIANTE tiene un papel central porque interactúa con

habilidades, intereses, grupos, eventos y otros estudiantes. Las relaciones SIGUE, CREA, PARTICIPA

y PERTENECE permiten representar acciones diferentes y hacen que el modelo se acerque al

funcionamiento real de una red social académica. Además, el análisis de las cardinalidades permitió comprender por qué algunas relaciones son 1: N

y otras N: M. También permitió reconocer la importancia de identificar correctamente las claves

primarias y de organizar los atributos para evitar duplicidad de información. Finalmente, considero que la elaboración de este modelo fortaleció mis conocimientos sobre el

diseño de bases de datos y me permitió comprender que un buen modelo entidad–relación no

consiste únicamente en conectar cuadros, sino en analizar cuidadosamente qué información se

necesita, cómo se relaciona y qué reglas deben cumplirse para que el sistema funcione de manera

organizada y coherente.

